

Laboratório de Concorrência · Aula 10

Danilo Antonio Alves Rosa · RA 24047644-2 · Turma B

Gerado em 09/10/2026, 20:36:24 · tempo registrado no cronômetro: 0:00

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

A sincronização de cooperação acontece quando uma tarefa precisa aguardar a finalização da outra,

Ex: Aguardar na fila do refeitório para esquentar no almoço no micro-ondas.

A sincronização de competição acontece quando as tarefas precisam utilizar o mesmo recurso, mas não podem utilizar ao mesmo tempo,

Ex: Duas pessoas querendo usar o banheiro ao mesmo tempo, mais cada uma deve aguardar sua vez.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

Espero que imprima 2.000.000

1ª execução (n = 1.000.000)

1344214

2ª execução (n = 1.000.000)

1019445

3ª execução (n = 1.000.000)

1028472

Saída do programa

não respondido

```
1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
2 // Cole em https://onecompiler.com/java
3 public class Main {
4     static int contador = 0; // variável COMPARTILHADA
5
6     public static void main(String[] args) throws InterruptedException {
7         int n = 1_000_000; // repetições de cada thread
8         Runnable incrementa = () -> {
9             for (int i = 0; i < n; i++) contador++;
10        };
11        Thread t1 = new Thread(incrementa);
12        Thread t2 = new Thread(incrementa);
13        t1.start(); t2.start(); // as duas executam ao mesmo tempo
14        t1.join(); t2.join(); // espera as duas terminarem
15        System.out.println("esperado: " + 2 * n);
16        System.out.println("obtido: " + contador);
17    }
18 }
```

Console

esperado: 2000000
obtido: 1028472

952 ms

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não bateram com o esperado, foram diferentes nas 3 execuções, mudaram de uma execução para outra pois ocorre uma condição de corrida entre as threads, que acessam a mesma variável sem sincronização

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

A atualização se perde pois as duas threads leem o mesmo valor de contador antes que alguma delas gravem o resultado,

Ler: a Thread 1 lê o valor 10 e a Thread 2 também lê 10.

Somar: as duas threads somam 1 ao valor lido, obtendo 11.

Gravar: a Thread 1 grava 11 e a Thread 2 também grava 11.

1.3 Com $n = 1000$, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Nesse caso, o resultado somo corretamente 1000 com 1000, porém mesmo assim continua sem sincronização. Esse erro é perigoso em sistemas reais porque pode ocorrer de forma imprevisível. Durante os testes, o programa pode apresentar o resultado correto, mas, em produção, com mais usuários e threads executando simultaneamente, pode apresentar resultados incorretos. Isso pode causar inconsistências em saldos bancários, estoques, contagem de acessos e outros dados compartilhados.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8

ABAABB → 6

ABBBAA → 4

BAAABB → 6

BBABAA → 4

BBBAAA → 7

Saída do programa

não respondido

```
8 def executa(ordem):
9     total = 3
10    reg = {"A": 0, "B": 0}
11    passo = {"A": 0, "B": 0}
12    for t in ordem:
13        if passo[t] == 0:
14            reg[t] = total
15        elif passo[t] == 1:
16            reg[t] = reg[t] + 1 if t == "A" else reg[t] * 2
17        else:
18            total = reg[t]
19            passo[t] += 1
20    return total
```

Console Output:

```
AAABBB -> 8
BBBAAA -> 7
ABABAB -> 6
Ordens possíveis: 20
TOTAL = 4: 9 ordens, por exemplo AABBBB
TOTAL = 6: 9 ordens, por exemplo AABABB
TOTAL = 7: 1 ordens, por exemplo BBBAAA
TOTAL = 8: 1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

Temos 20 ordens possíveis, com 4 resultados diferentes, apenas 1 ordem produz o resultado 8

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

ABBBAA:

A lê: a tarefa A lê TOTAL = 3 e guarda esse valor.

B lê: a tarefa B também lê TOTAL = 3.

B calcula: B multiplica 3 por 2 e obtém 6.

B grava: B grava TOTAL = 6.

A calcula: A soma 1 ao valor que havia lido inicialmente (3), obtendo 4.

A grava: A grava TOTAL = 4, substituindo o valor 6 que B havia gravado.

O que aconteceu com a operação de B? A atualização de B foi sobrescrita pela tarefa A. Como A utilizou o valor antigo que havia lido, gravou 4 em vez de preservar o resultado 6 de B. Por isso, o resultado final é 4, em vez de 8.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Creio que a operação atômica, pois essa é mais otimizada e rápida

Acho que todos realizarão corretamente

Saída do programa

não respondido



```
1 // Prática 3 - Corrigindo a corrida: monitor e operação atômica (Sebesta 13.4 e 13.7, p. 557-578)
2 // Cole em https://onecompiler.com/java
3 import java.util.concurrent.atomic.AtomicInteger;
4
5 public class Main {
6     static int semCuidado = 0, comSync = 0;
7     static AtomicInteger atomico = new AtomicInteger();
8
9     static synchronized void incSync() { comSync++; } // uma thread por vez
10
11     static long mede(Runnable corpo) throws InterruptedException {
12         Thread a = new Thread(corpo), b = new Thread(corpo);
13         long inicio = System.nanoTime();
14         a.start(); b.start(); a.join(); b.join();
15         return (System.nanoTime() - inicio) / 1_000_000; // milissegundos
16     }
17 }
```

Console Output:

Método	Tempo (ms)
esperado:	2000000
sem sincronizacao:	1046334 em 2 ms
synchronized:	2000000 em 83 ms
AtomicInteger:	2000000 em 71 ms

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

Monitor e operação atômica deram o valor correto, as versões corretas são mais lentas pois exigem mais processamento

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

não respondido

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

não respondido

Saídas (antes e depois da correção)

não respondido

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

não respondido

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

não respondido

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

não respondido

Tempo com 1 trabalhador

não respondido

Tempo com 3 trabalhadores

não respondido

Tempo com 6 trabalhadores

não respondido

Saída do programa

não respondido

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

não respondido

Saídas (antes e depois da troca)

não respondido

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

não respondido

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

não respondido

Saídas (o erro e o resultado com move)

não respondido

Saída do programa

não respondido

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

não respondido

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

não respondido

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

não respondido

Saída do programa

não respondido

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

não respondido

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

não respondido

Fechamento: Quem impede a corrida?

Java sem sincronização

não respondido

Java com synchronized

não respondido

Go com canais

não respondido

Rust com Arc e Mutex

não respondido

JavaScript

não respondido

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

não respondido

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

não respondido